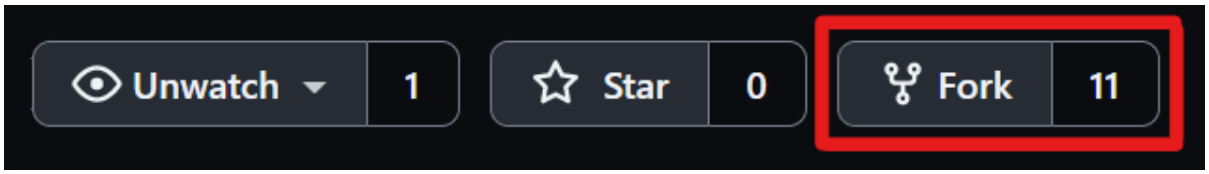
Forking a repository

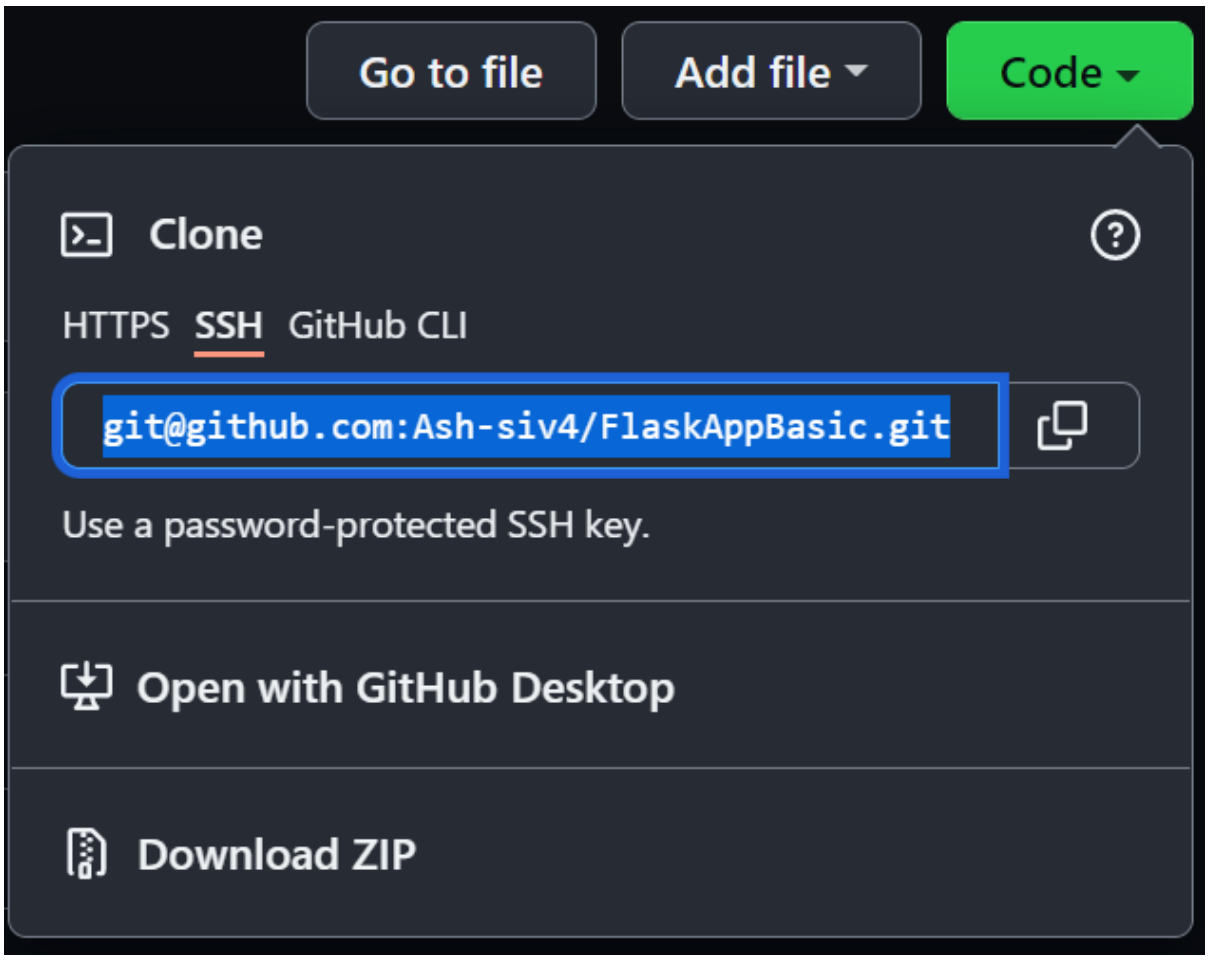
In this tutorial, you will fork a remote repository, configure upstream (to get updates from the original repository) and propose a change to the owner.

1. Make sure you're logged into your GitHub account.
2. Go to any public repository available. [This one]() for example.
3. To fork the repository, click on Fork (in the top-right of the page):



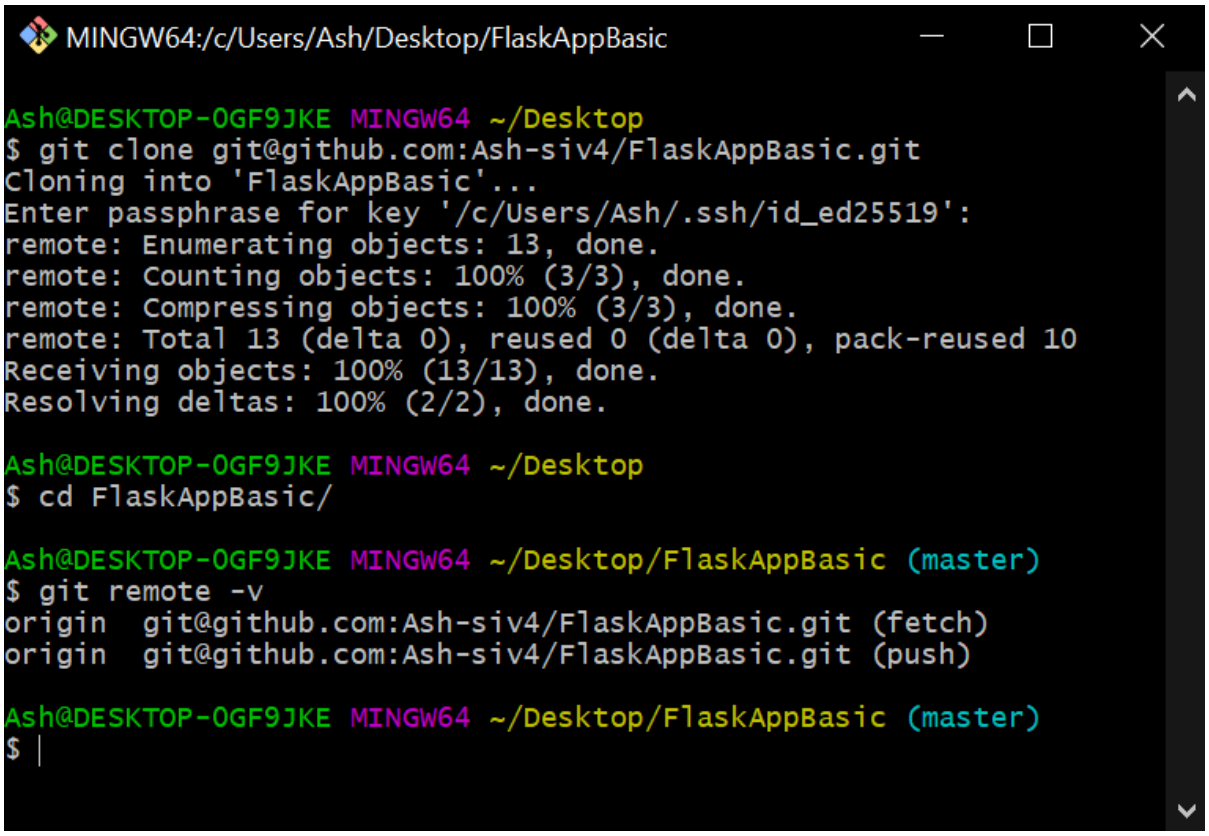
   This will create a copy of the repository under your account.

4. You will then be redirected to your account's version of the repository.
5. Click on the green `Code` button and **copy** the `SSH` URL.



6. Open a Git Bash terminal in the location you want to clone your project into.
7. Run the command `git clone [URL]`, but replace [URL] with the repository URL you copied in the previous step.
8. Change directory to the project you just cloned.
9. Run the following command, to confirm that you have cloned the correct repository: `git remote -v`.

   You should get a similar output for `fetch` and `push`, but the URL will be pointing to your repository.



Updating forked repository from original

Next, we'll set up your local project to receive updates from the original repository. This is required, so that when the owner of the original repository adds new functionality, such as bug fixes, you will be able to get these changes as well.

1. Open your Bash terminal, in the root directory of your project. You should be able to see that you're on the main branch.

   Next, you want to execute `git remote -v` to make sure that the upstream to the original repository is not set up yet.

   You should see a similar output to this:

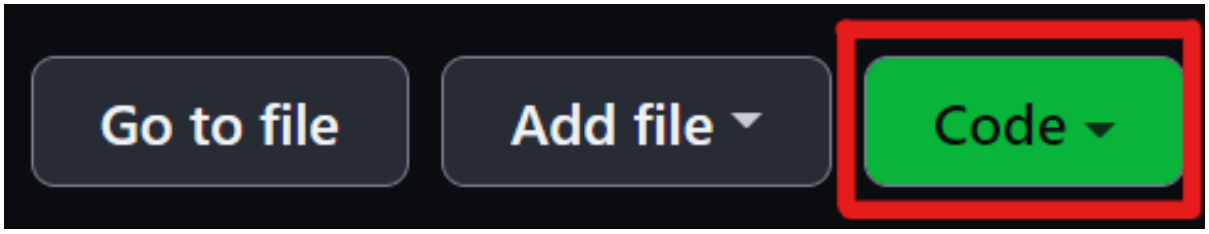
```
MINGW64:/c/Users/Ash/Desktop/FlaskAppBasic

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git remote -v
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (fetch)
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (push)

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ |
```

2. In your browser, go to the original git repository and copy the repository URL.

   You can do this by clicking on the green `Code` button and copying the `SSH` URL:



3. Execute the following command in your Bash terminal but replace the [URL] with the one you just copied: `git remote add upstream [URL]`.
4. Next, you want to check that this has worked, which you can do by executing the `git remote -v` command.

   The output should look similar to this:

```
MINGW64:/c/Users/Ash/Desktop/FlaskAppBasic

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git remote -v
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (fetch)
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (push)

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git remote add upstream git@github.com:tvaidotas/FlaskAppBasic.gi
t

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git remote -v
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (fetch)
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (push)
upstream        git@github.com:tvaidotas/FlaskAppBasic.git (fetch)
upstream        git@github.com:tvaidotas/FlaskAppBasic.git (push)

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$
```

5. Now you will pull the changes from the original repository into yours, which you can do by executing the `git fetch upstream` command in your Git Bash terminal.

   The output will depend on whether there are new changes or not:

```
MINGW64:/c/Users/Ash/Desktop/FlaskAppBasic

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git remote -v
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (fetch)
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (push)

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git remote add upstream git@github.com:tvaidotas/FlaskAppBasic.gi
t

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git remote -v
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (fetch)
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (push)
upstream        git@github.com:tvaidotas/FlaskAppBasic.git (fetch)
upstream        git@github.com:tvaidotas/FlaskAppBasic.git (push)

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git fetch upstream
Enter passphrase for key '/c/Users/Ash/.ssh/id_ed25519':
From github.com:tvaidotas/FlaskAppBasic
 * [new branch]      dependabot/pip/jinja2-2.11.3 -> upstream/dependa
bot/pip/jinja2-2.11.3
 * [new branch]      master     -> upstream/master

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$
```

6. You will want to update the main branch, so we will merge the *upstream/main* into the *local origin/main*.

   You can do this by executing the `git merge upstream/[name_of_main_branch` command.

   Example command: **git merge upstream/main**

   Keep in mind that there may be merge conflicts that you will need to resolve.

   If you have resolved the merge conflicts, or if there were none, you should *push* to update your *origin/main* (or *origin/master*) branch.

MINGW64:/c/Users/Ash/Desktop/FlaskAppBasic

```
Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git remote -v
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (fetch)
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (push)

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git remote add upstream git@github.com:tvaidotas/FlaskAppBasic.gi
t

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git remote -v
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (fetch)
origin  git@github.com:Ash-siv4/FlaskAppBasic.git (push)
upstream        git@github.com:tvaidotas/FlaskAppBasic.git (fetch)
upstream        git@github.com:tvaidotas/FlaskAppBasic.git (push)

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git fetch upstream
Enter passphrase for key '/c/Users/Ash/.ssh/id_ed25519':
From github.com:tvaidotas/FlaskAppBasic
 * [new branch]      dependabot/pip/jinja2-2.11.3 -> upstream/dependa
bot/pip/jinja2-2.11.3
 * [new branch]      master     -> upstream/master

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git merge upstream/master
Already up to date.

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$ git push
Enter passphrase for key '/c/Users/Ash/.ssh/id_ed25519':
Everything up-to-date

Ash@DESKTOP-OGF9JKE MINGW64 ~/Desktop/FlaskAppBasic (master)
$
```

Handling Merge Conflicts

You will now go through the steps required to cause a merge conflict.

Initialize a Repository for Testing Merge Conflicts

1. Open a terminal
2. Create a new directory by executing `mkdir git-merge-conflict`
3. Change directory by executing `cd git-merge-conflict`
4. Initialize this directory as a git repository by executing `git init .`
5. Create a new text file *hello.txt* in the directory
6. Add some text, such as "hello world", to the *hello.txt* file. Save and close the file
7. Now you need to make git track changes for the *hello.txt* file, which can be done by executing the `git add hello.txt` command
8. Now you need to create a save point, which is known as a commit. This will have the current state of the *hello.txt file in it.

   Execute the `git commit -m "initial commit"` command to achieve this.

Create a Branch with a Conflict

Now that you have a repository and a main branch with a file on it, the next step is to create a new branch to use to cause the merge conflict.

1. Checkout a new branch by executing `git checkout -b new-branch`
2. Open the text file *hello.txt* and add a new line of text to it, such as "making a change to the file". Your text file should now look like this:
3.
```
hello world
making a change to the file
```

4. Let's ensure Git keeps track of the `hello.txt` file that we made a change to. This can be done by executing the `git add hello.txt` command.

   Let's commit this change and add a message that reflects the change made i.e. - `git commit -m "made a change to hello.txt file"`. The change we made will now try to override the changes in **main** branch, for the text file `hello.txt`.

5. A change to the text file `hello.`txt is now required before we can cause a merge conflict. Let's go back to our **main** branch, by executing the `git checkout main` command.
6. Open the text file `hello.txt` and add a new line of text to it, such as **"making a bigger change"**.

Your text file on main should now look like this:

```
hello world
making a bigger change
```

The `hello.txt` on the **new-branch** branch should look like this:

```
hello world
making a change to the file
```

Let's commit the change we made to the `hello.txt` file to the main branch, by executing `git add hello.txt` followed by `git commit -m "modified hello.txt file"`.

Attempt to Merge the New Branch

Now, we want to merge changes from the *new-branch* branch to the *main* branch.

However, Git will not be able to figure out which version of the second line to use, creating a merge conflict, which the developer will be responsible for resolving.

1. Let's go ahead and actually cause the merge conflict, by executing the `git merge new-branch` command. You should get an output similar to this:

   ```
   Auto-merging hello.txt
   CONFLICT (content): Merge conflict in hello.txt
   Automatic merge failed; fix conflicts and then commit the result.
   ```

2. Let's take a look at the contents of the *hello.txt* file now, which should look similar to this:

   ```
   hello world
   <<<<<<< HEAD
   making a bigger change
   =======
   making a change to the file
   >>>>>>> new-branch
   ```

   The main thing to recognise is that the first line (hello world) doesn't have a conflict, but there is a conflict between the second line of the *main* and *new-branch* branches.

Resolving the Conflict

To resolve the merge conflict, there are a couple of steps needed:

1. Firstly, decide which second line to keep out of `making a bigger change` or `making a change to the file`. You could also choose something entirely different, such as keeping or deleting both lines.
2. Secondly, delete the lines *Git* added, to show where the merge conflict is happening i.e. `<<<<<<< HEAD`, `=======`, `>>>>>>> new-branch`.

   Let's say you decide to keep the second line that's currently in *main* branch. After cleaning up the file, it should look like this:

   ```
   hello world
   making a bigger change
   ```

3. Next, you need to save the changes made. This can be done by executing the `git add hello.txt` command, followed by the `git commit -m "resolved merge conflict"` command.

   Running `git status` should now indicate that there are no longer any conflicts to resolve.

Revert and Reset

1. Create a folder called "tmp"
2. Initialize the folder as a git repository.
3. Create a new file called `test.txt`.
4. Stage the file and commit, with an explicit commit message reflecting the additions made.
5. Repeat the previous two steps (using a different file name each time) **until you have done 5 commits**.
6. Check your `git log` history for the branch you are on (try out the additional flags for viewing the git log history).

At this stage, note down what you see in your history!

7. Use the `git revert HEAD` command to *undo* the last commit you did.
8. Use the ls command to see what happened.

Note down what you see in your directory!

9. Use the `reset` command to go back to your first commit.
10. Use the `ls` command to see what happened.

11. Check the history for the branch you're working on.

Github Reviews

NB. This task works much better with somebody else. If you really want to do it individually, you will need to make another GitHub account...

1. Create a Pull Request, similar to how you will have done in the Pull Requests module, to merge a feature change from a feature branch into the main branch (ideally, you will make something that requires changing - for example, some broken code).
2. Add a Reviewer to the PR.
3. Have the other person/other account do the same thing on their GitHub (requesting a review from you).
4. Start the review, making sure to test the new feature(s).
5. Make some comments for the other person/account.
6. Either approve the review, or request changes.
7. Complete the review.
8. Complete the merge once your own PR has been reviewed and approved by the other person/account.